

Рабочая тетрадь студента

Кружок: «Кодим под Android»

Тема занятия: Алгоритмы и блок-схемы

Продолжительность: 4 академических часа

Уровень: начинающий

ФИО студента: ____ Колесник Ирина _____

Группа: __1РКИ-3-26_____

Дата: ____8 октября 2026_____

Цель занятия

Научиться составлять алгоритмы и записывать их в виде блок-схем, чтобы в дальнейшем легко переводить их в код на Kotlin.

Что ты узнаешь сегодня

Как описывать порядок действий словами.

Как рисовать блок-схемы для разных видов алгоритмов.

Как проверять алгоритм на ошибки.

Зачем это нужно

Программист сначала **продумывает алгоритм**, а потом пишет код. Если алгоритм неправильный — программа не заработает, даже если код написан без ошибок.

Блок-схема помогает **увидеть логику** до того, как ты начнёшь писать код. На собеседованиях в IT-компаниях часто просят **объяснить алгоритм** решения задачи — без этого навыка не обойтись.

Часть 1. Теория (2 часа)

1.1. Алгоритм — это порядок действий

Алгоритм — это чёткая последовательность шагов для решения задачи.

Зачем это нужно: компьютер не умеет думать сам. Ему нужно точно сказать, что делать и в каком порядке. Если порядок нарушить — результат будет неправильным.

Пример: «Как открыть дверь ключом»

1. Достать ключ.
2. Вставить ключ в замок.
3. Повернуть ключ.
4. Нажать на ручку.
5. Открыть дверь.

Разбор: если поменять шаги 1 и 2 местами — вставить ключ, которого нет в руке, — не получится. Порядок важен.

1.2. Свойства алгоритма

Дискретность — состоит из отдельных шагов.

Понятность — каждый шаг ясен исполнителю.

Конечность — заканчивается за конечное число шагов.

Результативность — приводит к результату.

Массовость — подходит для похожих задач.

Зачем это нужно: если алгоритм непонятен, бесконечен или не даёт результата — это не алгоритм, а набор бесполезных действий. Проверяй свои алгоритмы по этим свойствам.

Пример нарушения понятности: «Сделай красиво» — непонятный шаг, потому что «красиво» каждый понимает по-своему.

1.3. Виды алгоритмов

Линейный — все шаги выполняются один раз по порядку.

Ветвящийся — есть выбор из двух вариантов по условию.

Циклический — некоторые шаги повторяются.

Комбинированный — сочетает несколько видов.

Зачем это нужно: в программировании эти виды превращаются в конкретные конструкции языка: линейный — в последовательные команды, ветвящийся — в `if/when`, циклический — в `for/while`. Поняв вид алгоритма, ты сразу знаешь, какую конструкцию использовать в коде.

1.4. Три способа записи алгоритма

Словесный — обычными предложениями.

Графический — блок-схемой.

Программный — на языке программирования.

Зачем это нужно: на занятии ты будешь использовать все три. Сначала — словами, потом — блок-схемой, потом — кодом. Так ты научишься переводить мысль в программу.

Пример одной и той же задачи тремя способами:

Словесно: «Если число больше нуля — вывести "плюс", иначе — "минус"».

Блок-схемой: ромб с условием и две ветки.

Программно:

```
kotlin
if (x > 0) {
    println("плюс")
} else {
    println("минус")
}
```

1.5. Элементы блок-схемы

Овал — начало или конец алгоритма.

Прямоугольник — действие.

Ромб — условие с двумя выходами: да и нет.

Параллелограмм — ввод или вывод данных.

Стрелка — показывает направление.

Зачем это нужно: блок-схема — это универсальный язык. Её поймёт программист на любом языке. Если ты умеешь рисовать блок-схему — ты сможешь объяснить свою идею кому угодно.

1.6. Как читать блок-схему

Движение всегда начинается с овала «Начало» и идёт по стрелкам.

В ромбе задаётся вопрос — от него два пути: «да» и «нет».

Овал «Конец» завершает алгоритм.

Зачем это нужно: читать чужой код сложно, а читать чужую блок-схему — легко. Это навык работы в команде.

1.7. Как проверить алгоритм

Тестирование — прогон на разных данных.

Трассировка — пошаговое выполнение с записью значений.

Проверка границ — поведение на крайних значениях.

Зачем это нужно: программист всегда проверяет свою программу. Если не проверить алгоритм на бумаге — придётся искать ошибку в коде, а это в разы дольше.

Пример проверки границ: если алгоритм проверяет «возраст ≥ 18 », обязательно проверь значения 17, 18 и 19 — именно на границах чаще всего бывают ошибки.

Проверь себя (устно)

Что означает ромб в блок-схеме?

Сколько выходов у условия?

Чем циклический алгоритм отличается от линейного?

Зачем проверять алгоритм на граничных значениях?

Часть 2. Практика (2 часа)

Задание 1. Составь словесный алгоритм (10 минут)

Зачем: научиться формулировать шаги чётко, чтобы их понял любой исполнитель.

Составь алгоритм «**Как помыть посуду**». Запиши 5–6 шагов.

Подсказка: начни с «взять губку» и не забудь про сушку.

Пример начала: 1. Открыть кран. 2. ...

1. Открыть кран
2. Взять губку
3. Взять средство для мытья посуды
4. Нанести средство на губку
5. Потереть посуду губкой со средством
6. Смыть средство с посуды

Вид алгоритма: ____Линейный____

Задание 2. Определи вид (10 минут)

Зачем: быстро распознавать вид алгоритма, чтобы выбрать правильную конструкцию в коде.

А) «Включить компьютер, ввести пароль, нажать Enter».

Ответ: __Линейный____

Б) «Если есть домашнее задание — сделать, иначе — отдохнуть».

Ответ: ____ Ветвящийся ____

В) «Повторять 5 раз: присесть и встать».

Ответ: ____ Циклический ____

Г) «Пока не выучишь стих, повторяй: прочитай, расскажи, проверь».

Ответ: ____ Комбинированный ____

Пример ответа на А: линейный, потому что шаги выполняются один раз по порядку.

Задание 3. Построй блок-схему с условием (20 минут)

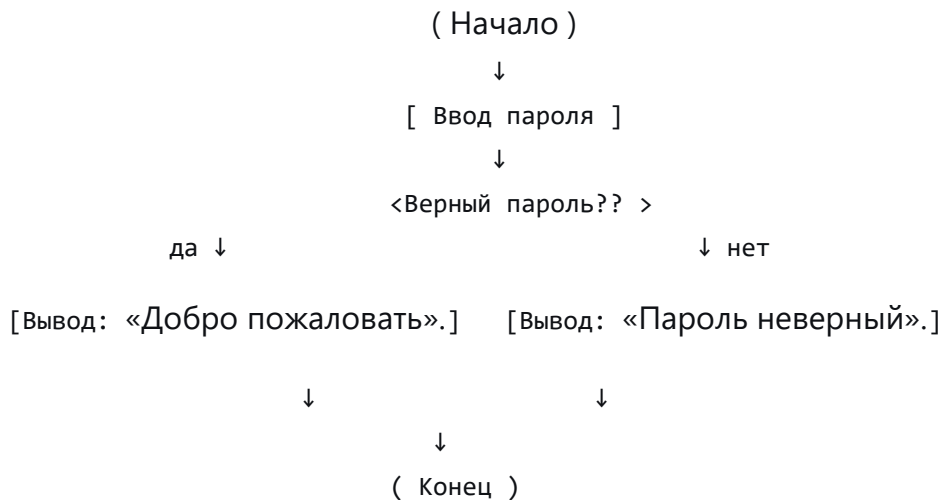
Зачем: научиться переводить условие «если..., иначе...» в графическую форму — это основа будущего `if/when` в Kotlin.

Алгоритм «Проверка пароля»:

Если пароль верный — вывести «Добро пожаловать».

Иначе — вывести «Пароль неверный».

Нарисуй блок-схему:



(Конец)

Подсказка: используй параллелограмм для ввода пароля, ромб для проверки и два прямоугольника для вывода.

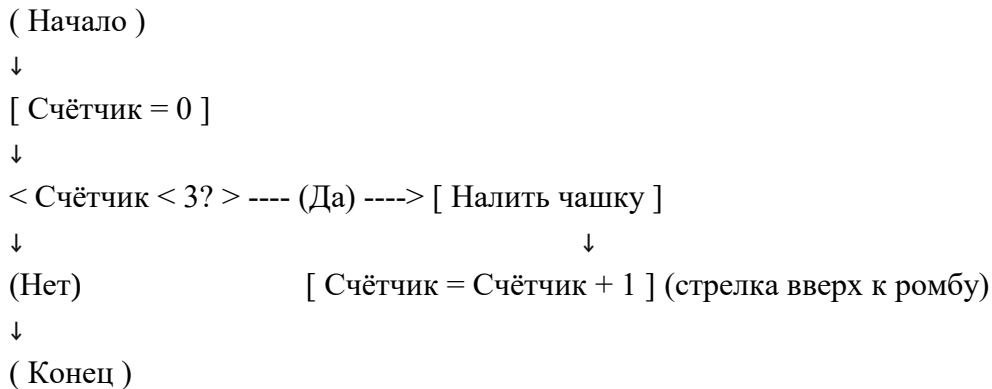
Задание 4. Построй блок-схему с циклом (20 минут)

Зачем: циклы в коде (for, while) — одни из самых сложных тем для новичков. Блок-схема помогает понять, что именно повторяется и когда цикл заканчивается.

Алгоритм «Налить 3 чашки чая»:

Пока чашек меньше 3 — налить чашку и увеличить счётчик.

Нарисуй блок-схему:



Подсказка: начало счётчика — прямоугольник, проверка условия — ромб, тело цикла — прямоугольник, возврат к проверке — стрелка вверх.

Задание 5. Трассировка (15 минут)

Зачем: уметь вручную «прогнать» алгоритм и получить результат. Этот навык нужен для отладки кода.

Дан алгоритм:

1. $X = 7$.
2. Если $X > 5$ — вывести «Много».
3. Иначе — вывести «Мало».
4. Конец.

Что выведет алгоритм? _____ много _____

А если $X = 5$? _____ мало _____

А если $X = 1$? _____ мало _____

Пример решения для $X = 7$: условие $7 > 5$ истинно, значит выводится «Много».

Подсказка для $X = 5$: проверь, строгое ли неравенство.

Задание 6. Найди ошибку (15 минут)

Зачем: в программировании ошибки в порядке шагов — самая частая причина, почему «ничего не работает». Учись видеть их сразу.

Алгоритм «Приготовить бутерброд»:

1. Съесть бутерброд.
2. Взять хлеб.
3. Намазать маслом.
4. Положить сыр.

Что не так? __все шаги выполнены не по порядку_____

Исправленный порядок:

1. Взять хлеб
2. Намазать маслом.
3. Положить сыр.
4. Съесть бутерброд

Подсказка: нельзя съесть то, что ещё не приготовлено.

Задание 7. Мини-проект (25 минут)

Зачем: собрать все навыки вместе — придумать алгоритм, определить вид, нарисовать блок-схему. Это то, что делает программист перед тем, как сесть за код.

Придумай свой алгоритм для задачи по выбору.

Например: «Как найти книгу в библиотеке», «Как записаться в кружок», «Как вызвать такси».

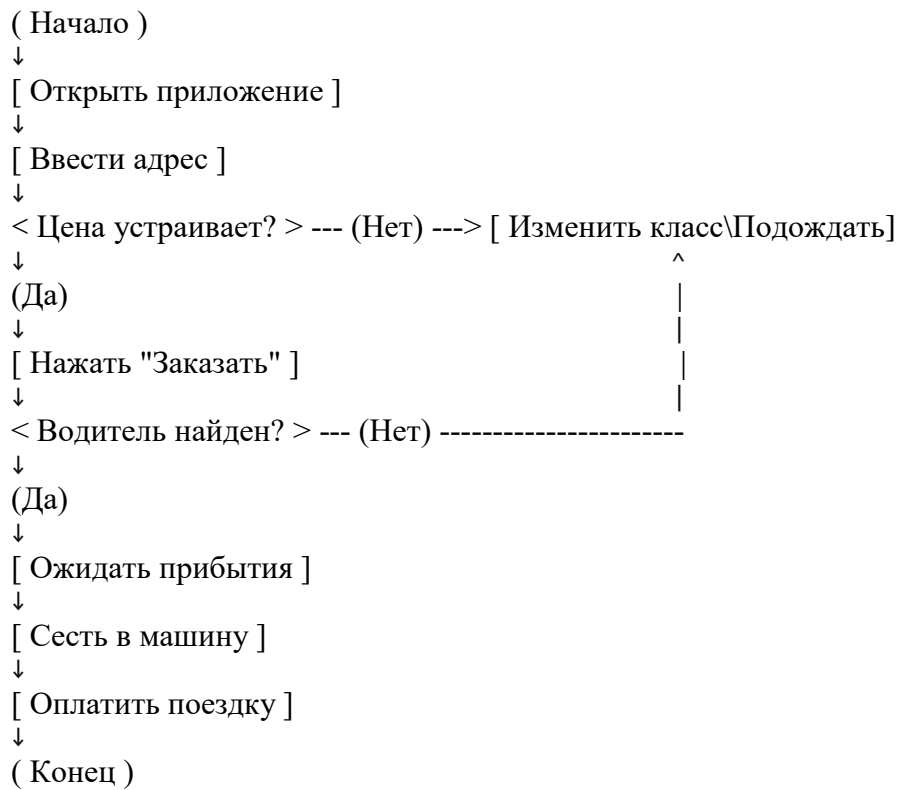
Название алгоритма: _____ «Как вызвать такси». _____

Шаги:

1. Открыть приложение
2. Ввести адрес
3. Цена устраивает?
4. Нет - Изменить класс\Подождать
5. Да - Нажать "Заказать"
6. Водитель найден?
7. Нет - Изменить класс\Подождать (шаг 4)
8. Да - Ожидать прибытия
9. Сесть в машину
10. Оплатить поездку

Вид алгоритма: _____ Ветвящийся _____

Блок-схема:



Итог занятия

Что я узнал(а):

Как описывать порядок действий словами.

Как рисовать блок-схемы для разных видов алгоритмов.

Как проверять алгоритм на ошибки.

Что осталось непонятным:

Что понравилось:

Всё

Домашнее задание

1. Составь алгоритм «Как добраться до колледжа» (5–6 шагов) и определи его вид.
2. Нарисуй блок-схему для алгоритма «Проверка оценки».

3. Придумай циклический алгоритм из повседневной жизни и нарисуй его блок-схему.

Оценка за занятие: _____

Подпись педагога: _____

Памятка. Элементы блок-схемы

Овал — начало или конец.

Прямоугольник — действие.

Ромб — условие.

Параллелограмм — ввод или вывод.

Стрелка — направление.

Важно:

Движение — только по стрелкам.

У ромба всегда два выхода: да и нет.

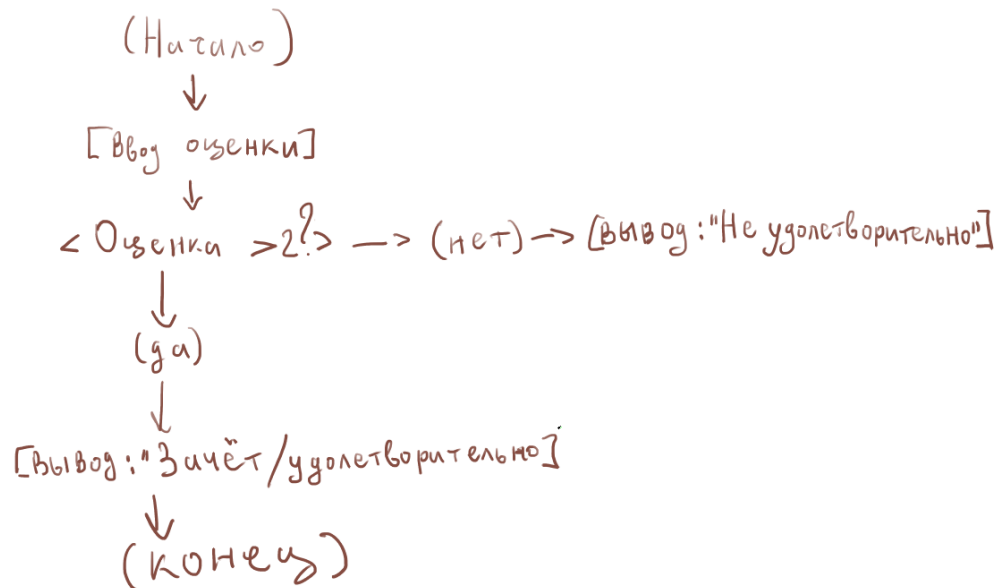
Цикл завершается, когда условие становится ложным.

Алгоритм заканчивается овалом «Конец».

**Рабочая тетрадь разработана для кружка «Кодим под Android»
ГБПОУ «ИТ.Москва», 2026 г.**

Домашнее задание

1. Составь алгоритм «Как добраться до колледжа» (5–6 шагов) и определи его вид.
 1. Выйти из дома.
 2. Дойти до остановки транспорта.
 3. Если автобус нужного маршрута уже подъехал — сесть в него.
 4. Иначе — ожидать следующего автобуса.
 5. Выйти на нужной остановке.
 6. Дойти до здания колледжа.
2. Нарисуй блок-схему для алгоритма «Проверка оценки».



3. Придумай циклический алгоритм из повседневной жизни и нарисуй его блок-схему.

(Начало)



[Поставить сырую пиццу в духовку]



<Она готова?> → (да) → (конец)



(нет)



[Оставить готовиться дальше]

